

Semantic Integration of Geospatial Data for Infrastructure Vulnerability Assessment

Roland Bernard, 19598, rolbernard@unibz.it

September 2026

Abstract

This report details a data curation pipeline developed to assess the exposure of critical infrastructure to natural hazards, specifically landslides and avalanches, within the Bolzano region. Utilizing spatial and administrative datasets derived from the CRIMA project, the project performs extensive data profiling, including single column analysis, unique column combination, functional dependency, and inclusion dependency discovery, using the Metanome platform and YData Profiling library. The extracted metadata directly guides the design of a relational schema which is subsequently deployed in a PostGIS database. To semantically integrate this data, the project introduces an OWL 2 QL ontology representing the domain of interest. Virtual Knowledge Graph mappings are designed via the Ontop plugin for Protégé, leveraging SQL joins and spatial PostGIS functions to dynamically compute topological intersections between infrastructure elements and hazard zones. Finally, the resulting knowledge graph is exposed through a custom React-based single-page web application. By querying the Ontop SPARQL endpoint, this interface enables users to interactively analyze and identify vulnerable infrastructure assets across specific municipalities, thereby supporting evidence-based repair and maintenance planning.

1 Introduction

This project focuses on the domain of natural hazards and their impact on exposed objects and infrastructure vulnerabilities. The aim is to identify and analyze critical infrastructure such as energy, water, electricity, and communications infrastructure that are geographically located within known hazard zones, such as zones prone to landslides and avalanches. Doing so could be helpful to assess potential vulnerabilities, identify likely points of failure, and estimate budgetary requirements for repairs.

By leveraging Virtual Knowledge Graphs, the project integrates raw infrastructure data, known hazard information, and administrative boundaries into a unified system. The resulting application is designed to process user parameters and answer complex spatial queries. Specifically, the application addresses questions such as which infrastructure asserts of a particular domain are located within high-risk landslide or avalanche zones. Such analysis could be of great value for identifying infrastructure lines which need more redundancy to avoid disruptions. The project also assesses municipal exposure by computing how aggregates across all or within specific municipalities. By determining which municipalities have the highest overall infrastructure exposure to natural hazards such insights could be used for planning required maintenance and repair budgets. Concretely, the resulting system is therefore able to directly answer the following types of queries:

- Which water-related infrastructure nodes are located within high risk landslide zones?
- What are the infrastructure lines that intersect with avalanche-prone areas?
- In a given Municipality, how many electricity lines are currently situated in a hazard zone?
- Which municipalities have the highest count of infrastructure elements exposed to natural hazards?

The remainder of this report is structured as follows: Section 2 gives an overview of the chosen data sources. Section 3 covers the data profiling part of the project, including single column analysis in Section 3.1, Section 3.2 covering unique column combinations, Section 3.3 containing analysis of functional dependencies, and Section 3.4 analyses inclusion dependencies. Section 3.5 describes the design and implementation of the relational database schema based on the preceding data profiling results. Section 4 covers the data integration part of the project, describing the developed ontology in Section 4.1, the virtual knowledge graph mappings in Section 4.2, and a description of the implemented application in Section 4.3. Finally, Section 5 concludes the report with a summary of the performed tasks.

2 Selected Data Sources

Following the project guidelines to integrate at least two different data sources, this project integrates five distinct datasets containing spatial and administrative information. As detailed in the guidelines, these data sources were introduced within the context of the CRIMA project. Some information about them was documented in the Teams channel of the Data Profiling module, specifically located in the folder `data/crima` project. Specifically, the integration will involve the following provided data sources:

- **bz_infrastructure_line**: This dataset provides geometric and categorical information regarding critical infrastructure lines. It contains multi-line geometries representing physical lines alongside classifications for infrastructure types, such as power lines and water pipes. The line dataset contains 47,551 rows detailing 11 types of line infrastructure.
- **bz_infrastructure_node**: This dataset contains point-like critical infrastructure information. It specifies the point geometries and infrastructure types for various nodes, such as water catchments, electrical substations, and communication infrastructure. The node dataset contains 4,223 rows representing 10 types of facilities.
- **bz_hazard_landslide**: This source supplies information regarding known landslide risk areas. It includes polygon geometries defining the spatial extent of the hazards, alongside categorical classifications of the landslide process types and level of danger. The landslide dataset contains 23,737 records categorized into four danger levels and five specific landslide process types.
- **bz_hazard_avalanche**: Similar to the landslide dataset, this source provides spatial boundaries and risk evaluations for avalanche-prone areas. It includes polygon geometries, avalanche process categorizations, and danger levels. The avalanche dataset includes 13,071 records, classified like the landslide dataset by danger severity and avalanche process type.
- **bz_admin_municipality**: This dataset provides the administrative context for the region. It contains 116 records including polygon geometries representing municipal boundaries, names in multiple languages, and district associations. This dataset allows the system to aggregate and assign hazard risks to specific municipalities.

Originally these datasets were derived from the Bolzano Province GeoKatalog. They provide the necessary geospatial geometries and categorical data required for the domain analysis. All the datasets were provided for this project in the form of GZip compressed CSV files. All datasets include geographic information stored in the Well-Known Text format. The files contain multiple data quality issues that will be identified and resolved as part of the data profiling part of this project.

3 Data Profiling

This section of the report discusses the data profiling part of the project. Section 3.1 covers the performed single column profiling steps, while Section 3.2, Section 3.3, and Section 3.4 will cover respectively the application of unique column combination, functional dependency, and inclusion dependency discovery. Finally, Section 3.5 covers the relational database schema designed for the data and how the profiling result affected design choices.

This part of the project made extensive use of the Metanome data profiling platform [8]. Each section will detail which Metanome algorithms were used. In addition to Metanome, also the YData Profiling Python library [15] has been used for single column analysis.

3.1 Single Column and Correlation Analysis

Single column analysis has been performed for this project using both the YData Profiling Python library and the SCDP algorithm implementation for Metanome. The goal of this analysis is primarily to understand the type and distribution of data contained within the individual columns. As such it will influence for example the data type chosen for the corresponding attributes in the relational schema and ontology developed in Section 3.5 and Section 4.1. Some correlation analysis has also been performed as part of this step using the YData Profiling library. Below I include, in addition to some commentary about the results, also tables that include the most relevant information for each column of the various tables.

3.1.1 bz_infrastructure_line and bz_infrastructure_node

bz_infrastructure_line Table 1 shows the most important single column analysis results for each column of the `bz_infrastructure_line` table. The table contains a total of 47551 rows. We can operate that the table contains

Column	Data Type	# NULLs	# Distinct	Min	Max	Avg.	Std.
ogc_fid	INT	0	47551	1	47551	23776	13726.94
gml_id	VARCHAR[48]	0	47551	-	-	-	-
objectid	REAL	0	47551	94188705	94462179	94295324.76	102812.43
code	REAL	0	11	1021101	1021111	1021103.63	2.42
codealt	-	47551	0	-	-	-	-
bez_i	VARCHAR[48]	0	11	-	-	-	-
bez_d	VARCHAR[48]	0	11	-	-	-	-
istat_code	REAL	6	115	21001	21118	21044.08	34.64
nda_link_de	VARCHAR[64]	6	115	-	-	-	-
nda_link_it	VARCHAR[64]	6	115	-	-	-	-
nda_text_de	VARCHAR[16]	6	1	-	-	-	-
nda_text_it	VARCHAR[16]	6	1	-	-	-	-
gem_text_d	VARCHAR[32]	6	115	-	-	-	-
gem_text_i	VARCHAR[32]	6	115	-	-	-	-
blp	-	47551	0	-	-	-	-
shape_len	REAL	0	45404	0.004	23001.3641	223.08	503.35
geom	TEXT	0	47115	-	-	-	-

Table 1: Single column analysis results for the different columns of the bz_infrastructure_line table extracted using the Metanome SCDP algorithm.

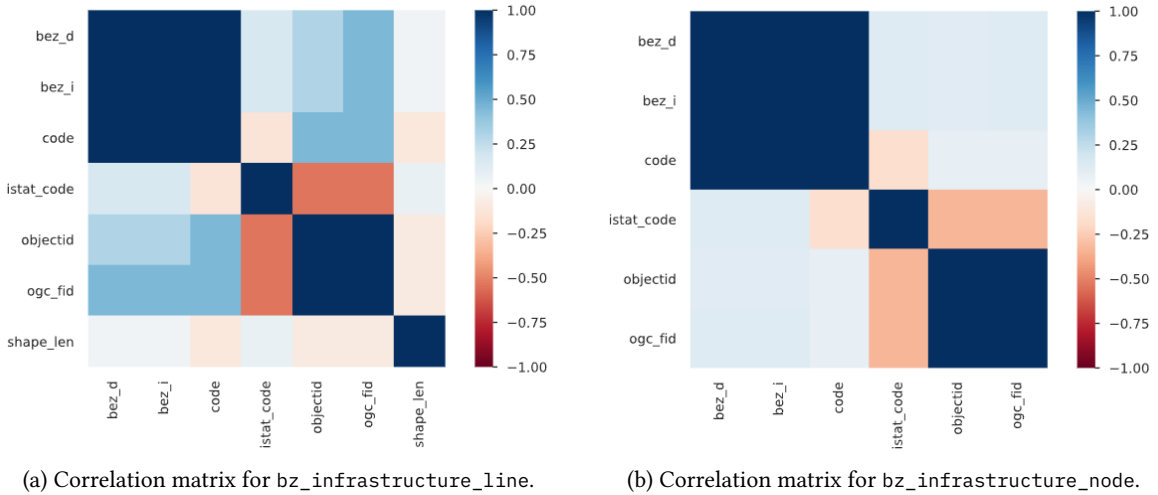


Figure 1: Column correlation matrix showing the correlation between all pairs of columns for the bz_infrastructure_line and bz_infrastructure_node tables. Spearman correlation is used for comparing numerical variables and Cramer’s V association coefficient for categorical variables.

three columns that individually form a unique key, ogc_fid, gml_id, and objectid. ogc_fid is a sequentially increasing integer, going from 0 to 47551. Interestingly, objectid is stored as a real data type, with values such as "94188705.0", "94188707.0", or "94188708.0". However, all values are whole integers. The values of gml_id all follow the pattern UrbanPlan-InfrastructurePlan-Network.<objectid> where <objectid> is the integer value stored in the objectid column of the same tuple. istat_code, nda_link_de, nda_link_it, gem_text_d, and gem_text_i all have 115 distinct values, indicating that they might contain related values. This relationship can also clearly be seen in the correlation matrix shown in Fig. 1a. Manually inspecting gem_text_d reveals that it contains the names of municipalities, presumably the one the infrastructure line belongs to. gem_text_i contains the Italian name and istat_code some kind of identifier. nda_link_de and nda_link_it contain links that when followed lead to the implementing rules to the land-use plan of the given municipality. Note that these municipality-related columns are each missing for six rows. code, bez_i, and bez_d all have 11 distinct values. These values seem to indicate the type of infrastructure that is represented by this tuple, with the values in bez_d being the German name, those in bez_i the Italian name, and the values in code being an integer identifier of the type. Note that like with

objectid, the values in code are represented as real numbers ending with ".0", meaning they all have integer values. The following are all possible values of bez_d: "Trinkwasserleitung", "Druckrohrleitung", "Abwasserleitung", "Schlammleitung", "Methangasleitung", "Hochspannungsleitung", "Mittelspannungsleitung", "Fernwärmeleitung", "Hochspannungsleitung - Bannstreifen", "Sonstige Leitungen", "Methangasleitung - Bannstreifen". Finally, geom contains the geometry of the infrastructure line. Manual analysis reveals that the values are in the Well-Known Text format [4] and all of them are of type MULTILINESTRING. shape_len seems to contain real values. Comparing them with the geometry in geom seems to indicate that shape_len might be the length of the infrastructure line. nda_text_de and nda_text_it have a constant value of "Öffne" and "Apri" respectively. codealt and blp only contain missing values and are therefore not particularly interesting for this project.

Column	Data Type	# NULLs	# Distinct	Min	Max	Avg.	Std.
ogc_fid	SMALLINT	0	4223	1	4223	2112	1219.22
gml_id	VARCHAR[48]	0	4223	-	-	-	-
objectid	REAL	0	4223	19130422	19222933	19163811.41	37377.07
code	REAL	0	10	1021201	1021211	1021203.73	3.58
codealt	-	4223	0	-	-	-	-
bez_i	VARCHAR[48]	0	10	-	-	-	-
bez_d	VARCHAR[32]	0	10	-	-	-	-
istat_code	REAL	0	115	21001	21118	21056.76	35.26
nda_link_de	VARCHAR[64]	0	115	-	-	-	-
nda_link_it	VARCHAR[64]	0	115	-	-	-	-
nda_text_de	VARCHAR[16]	0	1	-	-	-	-
nda_text_it	VARCHAR[16]	0	1	-	-	-	-
gem_text_d	VARCHAR[32]	0	115	-	-	-	-
gem_text_i	VARCHAR[32]	0	115	-	-	-	-
blp	-	4223	0	-	-	-	-
geom	VARCHAR[48]	0	4214	-	-	-	-

Table 2: Single column analysis results for the different columns of the bz_infrastructure_node table extracted using the Metanome SCDP algorithm.

bz_infrastructure_node The table bz_infrastructure_node contains columns that are very similar to those found in the bz_infrastructure_line table, as can be seen in Table 2. Like the table on infrastructure lines, here we also have the attributes ogc_fid, gml_id, and objectid, all of which are individually unique for all 4223 rows of the table. objectid still is an integer saved using a real number representation, and gml_id follows the pattern UrbanPlan-InfrastructurePlan-Primary.<objectid>. The variables code, bez_d, and bez_i now have only 10 distinct values, and instead of representing a type of infrastructure line, they represent the type infrastructure node. The following are all 10 distinct values for bez_d: "Wasserfassung", "Kommunikationsinfrastruktur", "Umspannstation", "Kläranlage", "E-Werk", "Recyclinganlage", "Reservoir", "Fernheizanlage", "Pumpstation", "Müllentsorgungsanlage". istat_code, nda_link_de, nda_link_id, nda_text_de, nda_text_it, gem_text_d, gem_text_i, codealt, blp have exactly the same values and properties as explained above for the bz_infrastructure_line table. Finally, the column geom still contains geographic information in the Well-Known Text format, however for this table all of them have type POINT instead of a multi-line.

3.1.2 bz_hazard_landslide and bz_hazard_avalanche

bz_hazard_landslide Table 3 shows the single columns analysis results for the bz_hazard_landslide table. The table contains 23737 rows, no missing values in any of the columns, and, like the previously analyzed tables, there are three columns that contain different types of identifiers: ogc_fid, gml_id, and objectid. ogc_fid is still a dense integer identifier, while objectid is a real formatted sparse integer identifier. gml_id strictly follows the pattern UrbanPlan-HazardZonePlan-Landslides.<objectid> similar to what has been seen for the infrastructure tables. The three columns istat_code, name_i, and name_d all have exactly 104 distinct values and perfect correlation, showing that these are related. Analysis of the values shows that these represent the municipality that contains the given hazard area. Interestingly, there is also gem_id which given the name could also represent the municipality, but it contains fewer distinct value. It can be seen from the data that actually there is a relationship between gem_id and istat_code, where gem_id is given by taking the last two decimal digits of the integer value of istat_code. There are also three columns, status_gzp, desc_i, and desc_d, that contain

Column	Data Type	# NULLs	# Distinct	Min	Max	Avg.	Std.
ogc_fid	SMALLINT	0	23737	1	23737	11869	6852.43
gml_id	VARCHAR[48]	0	23737	-	-	-	-
objectid	REAL	0	23737	24651048	24687744	24663401.68	7600.10
istat_code	REAL	0	104	21001	21118	21054.49	33.69
gem_id	REAL	0	89	0	99	41.25	28.57
name_i	VARCHAR[32]	0	104	-	-	-	-
name_d	VARCHAR[32]	0	104	-	-	-	-
status_gzsp	SMALLINT	0	1	1	1	1	0
desc_i	VARCHAR[16]	0	1	-	-	-	-
desc_d	VARCHAR[16]	0	1	-	-	-	-
code	REAL	0	4	1040101	1040104	1040102.12	1.05
pericolo	VARCHAR[48]	0	4	-	-	-	-
gefahr	VARCHAR[48]	0	4	-	-	-	-
id_process	VARCHAR[16]	0	6	-	-	-	-
processo	VARCHAR[32]	8	5	-	-	-	-
prozess	VARCHAR[16]	8	5	-	-	-	-
id_gp	VARCHAR[16]	0	11	-	-	-	-
id_gs	VARCHAR[16]	0	2	-	-	-	-
gradodistudio	VARCHAR[16]	1	2	-	-	-	-
bearbeitungstiefe	VARCHAR[16]	1	2	-	-	-	-
x_label	REAL	0	23575	607326.28	762672.42	685127.92	29383.36
y_label	REAL	0	23576	5121372.99	5219039.48	5168582.58	17790.22
geom	TEXT	0	23624	-	-	-	-

Table 3: Single column analysis results for the different columns of the bz_hazard_landslide table extracted using the Metanome SCDP algorithm.

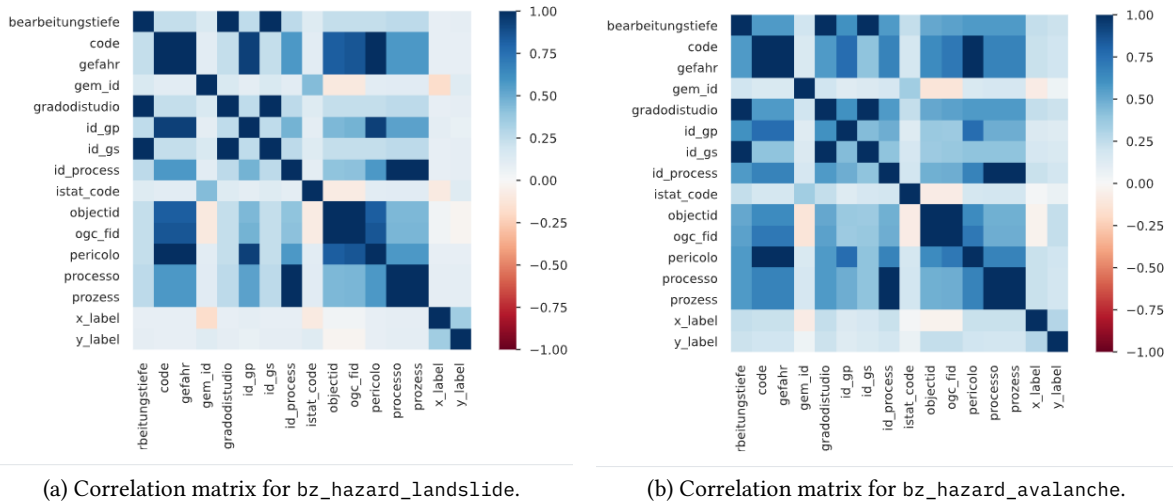


Figure 2: Column correlation matrix showing the correlation between all pairs of columns for the bz_hazard_landslide and bz_hazard_avalanche tables. Correlation is computed as explained for Fig. 1

only a constant value of respectively "1", "Approvato", and "Genehmigt". Further, another three related columns, code, pericolo, and gefahr contain the danger expressed as one of four different levels. The four distinct values for gefahr are "Massenbewegung - Untersucht und nicht gefährlich", "Massenbewegung - Mittlere Gefahrenstufe (H2)", "Massenbewegung - Hohe Gefahrenstufe (H3)", and "Massenbewegung - Sehr hohe Gefahrenstufe (H4)". Looking at Fig. 2a we can also see that there exists a column id_gp that is highly correlated with the danger level related attributes. It contains integer values from 0 to 9 as well as the value "n", the meaning of which is however unclear. There are also three variables, id_process, processo, and prozess, related to the type of landslide represented by the row. The five distinct values for prozess are "Sturz", "Massenbewegung", "Rutschung",

"Hangmure", and "Einbruch". Note that for some rows these contain missing values. Next, the table also contains information about the depth of the investigation performed in `id_gs`, `gradodistudio`, and `bearbeitungstiefe`, where all the distinct values for `bearbeitungstiefe` are "Mittlere (BT10)" and "Hoch (BT05)". The `geom` variable, like in all the other tables contains the geometry in Well-Known Text Format. All the geometries in this table are of type POLYGON. Finally, there are the columns `x_label` and `y_label` that contain real numbers. It is unclear what these value mean exactly, and they are uncorrelated to all other columns in the table.

Column	Data Type	# NULLs	# Distinct	Min	Max	Avg.	Std.
<code>ogc_fid</code>	SMALLINT	0	13071	1	13071	6536	3773.42
<code>gml_id</code>	VARCHAR[48]	0	13071	-	-	-	-
<code>objectid</code>	REAL	0	13071	13254987	13278119	13261982.19	4763.84
<code>istat_code</code>	REAL	0	104	21001	21118	21057.48	34.20
<code>gem_id</code>	REAL	0	89	0	99	42.46	30.14
<code>name_i</code>	VARCHAR[32]	0	104	-	-	-	-
<code>name_d</code>	VARCHAR[32]	0	104	-	-	-	-
<code>status_gzpz</code>	SMALLINT	0	1	1	1	1	0
<code>desc_i</code>	VARCHAR[16]	0	1	-	-	-	-
<code>desc_d</code>	VARCHAR[16]	0	1	-	-	-	-
<code>code</code>	REAL	0	4	1040301	1040304	1040301.90	1.26
<code>pericolo</code>	VARCHAR[64]	0	4	-	-	-	-
<code>gefahr</code>	VARCHAR[48]	0	4	-	-	-	-
<code>id_process</code>	VARCHAR[16]	0	5	-	-	-	-
<code>processo</code>	VARCHAR[32]	0	5	-	-	-	-
<code>prozess</code>	VARCHAR[16]	0	5	-	-	-	-
<code>id_gp</code>	VARCHAR[16]	0	11	-	-	-	-
<code>id_gs</code>	VARCHAR[16]	0	3	-	-	-	-
<code>id_phen</code>	-	13071	0	-	-	-	-
<code>gradodistudio</code>	VARCHAR[16]	1	2	-	-	-	-
<code>bearbeitungstiefe</code>	VARCHAR[16]	1	2	-	-	-	-
<code>x_label</code>	REAL	0	13039	607410.84	762672.42	682427.64	35643.06038
<code>y_label</code>	REAL	0	13035	5121372.99	5219039.48	5173225.75	18780.81
<code>geom</code>	TEXT	0	13050	-	-	-	-

Table 4: Single column analysis results for the different columns of the `bz_hazard_avalanche` table extracted using the Metanome SCDP algorithm.

bz_hazard_avalanche The table `bz_hazard_avalanche` contains 13071 rows and has a very similar structure to that of `bz_hazard_landslide`, as can be seen in Table 4. Most of the columns, including those related to the municipality, investigation depth, and geometry, are identical to those in `bz_hazard_landslide`. The identifiers have the same format, except for `gml_id` which here follows the pattern `UrbanPlan-HazardZonePlan-Avalanches.<objectid>`. While the overall danger classifications are similar, they have slight different naming, with the 4 distinct values in `gefahr` of this table being "Lawine - Untersucht und nicht gefährlich", "Lawine - Mittlere Gefahrenstufe (H2)", "Lawine - Hohe Gefahrenstufe (H3)", and "Lawine - Sehr hohe Gefahrenstufe (H4)". Like `bz_hazard_landslide`, this table also contains information about the different types of avalanches and the five distinct values in `prozess` are "Lawine", "Massenbewegung", "Fließlawine", "Staublawine", and "Gleitschnee". All other columns are exactly as described for `bz_hazard_landslide` above.

3.1.3 bz_admin_municipality

The `bz_admin_municipality` table contains 116 rows of data and its single column analysis results are shown in Table 5. This table contains numerous individually unique columns. The column `ogc_fid`, like the columns of the same name in the other tables, is a dense integer valued identifier. `id` also contains the exact same identifier as `ogc_fid`, however, it is formatted as a real value. `gml_id` is formatted as `Municipalities.<id>`, where `<id>` is the integer value of `id` for the respective row. The columns `code` and `istat_code` are also closely related, as can also be seen in the correlation matrix in Fig. 3. Both of these, while they are formatted as real values, contain only integer values. Further, executed tests show that `code` is always equal to the last three digits of the integer

Column	Data Type	# NULLs	# Distinct	Min	Max	Avg.	Std.
ogc_fid	SMALLINT	0	116	1	116	58.5	33.63
gml_id	VARCHAR[32]	0	116	-	-	-	-
id	REAL	0	116	1	116	58.5	33.63
code	REAL	0	116	1	118	59.08	34.34
istat_code	REAL	0	116	21001	21118	21059.08	34.34
zip_code	REAL	0	50	39010	39100	39032.48	14.64
belfiore_code	VARCHAR[16]	0	116	-	-	-	-
name_it	VARCHAR[32]	0	116	-	-	-	-
name_de	VARCHAR[32]	0	116	-	-	-	-
name_ld	VARCHAR[16]	107	9	-	-	-	-
distr_code	REAL	0	8	1	8	4.45	2.55
distr_de	VARCHAR[32]	0	8	-	-	-	-
distr_it	VARCHAR[32]	0	8	-	-	-	-
area	REAL	0	116	1660704.679	302498146.7	63786681	57613893.65
health_region	REAL	0	4	1	4	2.32	1.11
health_district	REAL	0	20	11	44	26.39	10.51
map_label	VARCHAR[64]	0	116	-	-	-	-
geom	TEXT	0	116	-	-	-	-

Table 5: Single column analysis results for the different columns of the bz_admin_municipality table extracted using the Metanome SCDP algorithm.

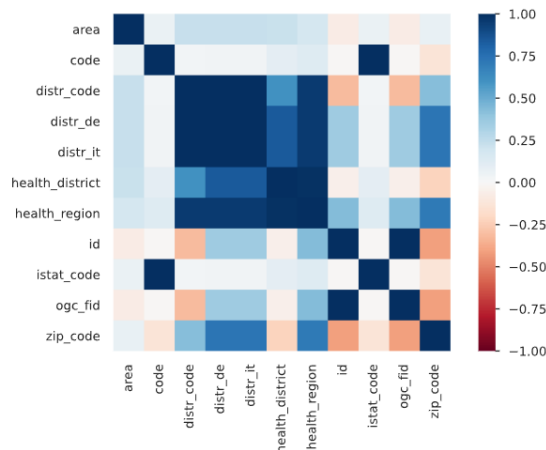


Figure 3: Column correlation matrix showing the correlation between all pairs of columns for the bz_admin_municipality table. Correlation is computed as explained for Fig. 1

value stored in istat_code. Another type of identifier, formed by one letter followed by three digits, is stored in belfiore_code. The name of the municipality is stored in three different languages in name_it, name_de, and name_ld. Note that while all rows contain values for the first two, the latter has many missing values is present only in 9 rows. Related to the name of the municipality, map_label contains a combination of the names in the different languages. The ZIP code of the municipality is stored in the zip_code columns. Again, while this is formatted as a real number, all of them contain a trailing ".0" meaning they represent integer values. Next, the district to which the municipality belongs is also stored in the columns distr_code, distr_de, distr_it. The eight distinct values for distr_code are the integers from 1 to 8, while the corresponding names for the district in distr_de are "Burggrafenamt", "Salten-Schlern", "Überetsch-Südt.Unterland", "Pustertal", "Wipptal", "Eisacktal", "Vinschgau", and "Bozen". The health region and district are also available respectively in health_region and health_district. The columns only contain integer values, again formatted as real values, and no information about the names of these is available in the used data sources. Finally, area contains the area of the covered by the municipality and geom contains a geometry of type POLYGON.

3.2 Unique Column Combination Discovery

Unique column combination discovery has been performed on all tables for this project using the Metanome HyUCC algorithm. This algorithm implements a version of the hybrid unique column discovery algorithm described in [10]. For the purposes of this analysis, NULL values have been configured to compare equal. The primary goal of this analysis is to identify possible keys in the dataset that can later be used as primary keys or regular unique constraints in the relational schema for Section 3.5. Note that all discovered minimal unique column families are single columns, so they have largely already been discussed in Section 3.1.

3.2.1 bz_infrastructure_line and bz_infrastructure_node

In both `bz_infrastructure_line` and `bz_infrastructure_node` there are only three minimal unique column combinations, and all of them consist of only single columns. As such, these have already been identified in Section 3.1.1. The minimal unique column combinations are `{ogc_fid}`, `{gml_id}`, and `{objectid}` for both of these tables. All three of these are candidates for being a primary key in the relational schema. However, `gml_id` is a string that can be generated easily from `objectid`, which is an integer formatted as a real value, as explained in Section 3.1.1. `objectid` might be used elsewhere in data sets not used for this project as a foreign key, however, for this project `ogc_fid` seems like the best candidate for the primary key, as it is a dense integer identifier.

3.2.2 bz_hazard_landslide and bz_hazard_avalanche

The results for the tables `bz_hazard_landslide` and `bz_hazard_avalanche` are very similar to those for the infrastructure data explained in Section 3.2.1. For both of these tables, the set of minimal unique column combination contains only the following single column combinations: `{ogc_fid}`, `{gml_id}`, and `{objectid}`. Again, all three of these are possible candidates for the primary key, however, following the same rational as in Section 3.2.1, the `objectid` column is likely the most appropriate choice for the primary keys of these two tables. `objectid` would be a more appropriate choice if we needed to use it as a foreign key in a different table, which is however not the case for this project.

3.2.3 bz_admin_municipality

Finally, the last table has a slightly different set of minimal unique column combinations compared to the other four. Still, all the minimal unique column combinations contain only a single column. Set full set of these minimal unique column combinations for the `bz_admin_municipality` table are `{ogc_fid}`, `{gml_id}`, `{id}`, `{code}`, `{istat_code}`, `{belfiore_code}`, `{name_it}`, `{name_de}`, `{area}`, `{map_label}`, and `{geom}`. From these, many are possible candidates for use as a primary key. However, `geom` and especially `area` do not make sense as primary keys. The latter is not necessary guaranteed to be unique in general outside this dataset. On the other hand, the former might technically be guaranteed to be unique in all cases, but even in that scenario it makes little practical sense to identify municipalities by their geometry. `name_it`, `name_de`, and `map_label` could technically be primary keys, but because there is no real way to choose a canonical language for the name, it is preferable to choose one of the identifiers as the primary key. From the remaining unique columns, which are all valid candidates for primary keys, `istat_code` is likely the best candidate for the primary key, because it is present also in the other tables as a foreign key.

3.3 Functional Dependency Discovery

Metanome using the HyFD algorithm implementation has been used for the discovery of Functional dependencies in this project. This implementation is based on the hybrid approach introduced in [11]. As with the unique column combination discovery, also for this analysis NULL values have been configured to compare equal. The results of this analysis can be used to identify parts of the tables that can possibly be extracted into separate relations in the design of the relational schema for Section 3.5.

3.3.1 bz_infrastructure_line and bz_infrastructure_node

bz_infrastructure_line Table 6 shows all the minimal functional dependencies discovered by the applied algorithm in the table `bz_infrastructure_line`. Aside from the two columns, `codealt` and `blp`, that are always missing and the three columns, `ogc_fid`, `gml_id`, and `objectid`, that are unique, we can still observe additional interesting information from the other functional dependencies. First, it can be seen that `geom` functionally determines many other columns. This is quite intuitive because the geometry of two distinct infrastructure lines

Determinant	Dependants
{}	codealt, blp
{ogc_fid}	*
{gml_id}	*
{objectid}	*
{geom}	istat_code, nda_link_de, nda_link_it, gem_text_d, gem_text_i, nda_text_de
{shape_len}	nda_text_de, nda_text_it
{istat_code}	nda_link_de, nda_link_it, gem_text_d, gem_text_i, nda_text_de, nda_text_it
{nda_link_de}	istat_code, nda_link_it, gem_text_d, gem_text_i, nda_text_de, nda_text_it
{nda_link_it}	istat_code, nda_link_de, gem_text_d, gem_text_i, nda_text_de, nda_text_it
{gem_text_d}	istat_code, nda_link_de, nda_link_it, gem_text_i, nda_text_de, nda_text_it
{gem_text_i}	istat_code, nda_link_de, nda_link_it, gem_text_d, nda_text_de, nda_text_it
{code}	bez_i, bez_d
{bez_i}	code, bez_d
{bez_d}	code, bez_i
{nda_text_de}	nda_text_it
{nda_text_it}	nda_text_de

Table 6: Functional dependencies discovered in bz_infrastructure_line table using the Metanome HyFD algorithm. Multiple columns in the dependents means that all of them are functionally dependent on the determinant. Further, unique column combinations have been marked with *.

is likely not to be the same, unless they go along the same route. The functional dependency shows that all rows with the same geometry belong to the same municipality. The functional dependency from shape_len to nda_text_de and nda_text_it, as well as those between nda_text_de and nda_text_it in both directions, can be explained by the fact that these two columns are constant except for the fact that they are missing in some rare cases. Next, it can be observed that there are a number of functional dependencies related to individual municipalities. Municipalities can be identified by one of istat_code, nda_link_de, nda_link_it, gem_text_d, or gem_text_i. When a municipality is given, then in addition to these identifying columns also nda_text_de and nda_text_it are known to be present. This suggests a relational schema where these municipality related variables are extracted into a separate relation. This relation would most appropriately be use the istat_code as a primary key. Finally, like in the case of the municipality, there are also three columns, code, bez_i, and bez_d, that all identify the kind of infrastructure that is represented by a row. These could also be extracted into separate relation with code as the primary key.

bz_infrastructure_node The functional dependencies discovered in the bz_infrastructure_node table are shown Table 7. Largely the picture is very similar to that seen for the table bz_infrastructure_line. Since this table is not missing municipality information for any row, the columns nda_text_de and nda_text_it are both also constant. Interestingly, in this case the geometry in geom functionally identifies all other variables except for the identifiers. This indicates two distinct infrastructure nodes can be at the same location and with the same properties. Lastly, the functional dependencies related to municipalities and infrastructure type are present and the same as those that have been identified and explained above for the table bz_infrastructure_line.

3.3.2 bz_hazard_landslide and bz_hazard_avalanche

bz_hazard_landslide For the table bz_hazard_landslide a very large number of 255 functional dependencies has been discovered. However, many of them are not particularly meaningfully, because they use one or more from geom, x_label, and y_label. These columns are related to the location of the hazard area, and as such are already nearly unique. This means that adding additional columns quickly indices functional dependencies. For this reason, these functional dependencies have been removed from Table 8. In the remaining functional dependencies, aside from the unique or constant columns, a number of different interesting observations can be made. First, like with the infrastructure tables, this table contains information about the municipalities that are made up of the columns istat_code, name_i, name_d, and gem_id. Note that all of these except gem_id could be used to identify the municipality. A similar relationship exists between the columns for the danger level, with the columns code, pericolo, and gefahr each being able to functionally determine the values of the others. Regarding the type of landslide, there is a similar relationship between processo and prozess. However, in this case there is some inconsistency with regard to the id_process. Interactive analysis the of these columns shows

Determinant	Dependants
{}	codealt
{}	nda_text_de
{}	nda_text_it
{}	blp
{ogc_fid}	*
{gml_id}	*
{objectid}	*
{geom}	istat_code, nda_link_de, nda_link_it, gem_text_d, gem_text_i, code, bez_i, bez_d
{istat_code}	nda_link_de, nda_link_it, gem_text_d, gem_text_i
{nda_link_de}	istat_code, nda_link_it, gem_text_d, gem_text_i
{nda_link_it}	istat_code, nda_link_de, gem_text_d, gem_text_i
{gem_text_d}	istat_code, nda_link_de, nda_link_it, gem_text_i
{gem_text_i}	istat_code, nda_link_de, nda_link_it, gem_text_d
{code}	bez_i, bez_d
{bez_i}	code, bez_d
{bez_d}	code, bez_i

Table 7: Functional dependencies discovered in bz_infrastructure_node table using the Metanome HyFD algorithm. Multiple columns in the dependents means that all of them are functionally dependent on the determinant. Unique column combinations have been marked with * as dependents for brevity.

that the reason for why this ID does not functionally determine the other two columns is that the values for processo and prozess are missing in very few cases. In these case a value for id_process is still present and not always the same. A very similar situation is also present for the investigation depth, where gradodistudio, bearbeitungstiefe, and id_gs are related. However, because there are is one row with missing values for gradodistudio and bearbeitungstiefe but not id_gs, the latter does not functionally determine the former two.

bz_hazard_avalanche The functional dependencies for table bz_hazard_avalanche are show in Table 9. Again, because in total the algorithm fount 281 different functional dependencies, Table 9 removes some of them that contain geom, x_label, or y_label that are largely not insightful. The overall structure of the functional dependencies is very similar to that for the landslide hazard zone dataset. The main difference is that because there are fewer missing values, it does not contain the misleading functional dependencies related to id_process and id_gs. The columns status_gzp, desc_i, desc_d, and id_phen are all constant and the columns ogc_fid, gml_id, and objectid are all individually unique in the dataset. The columns related to municipalities can clearly be identified as istat_code, name_i, and name_d. Each of these individually functionally determines the remaining two. Similarly, the same dynamic exists for the avalanche type represented by id_process, processo, and prozess. Further also the columns code, pericolo, and gefahr have the same relationship for the danger level. Finally, id_gs, gradodistudio, and bearbeitungstiefe are all information about the investigation depth.

3.3.3 bz_admin_municipality

All the functional dependencies discovered by the algorithm in the bz_admin_municipality table are shown in Table 10. This table, as has already been discussed in Section 3.2, contains numerous unique columns. Aside from the functional dependencies for these unique columns combinations, there are also two other interesting insights. The column health_district functionally determines the value of the health_region variable. Finally, the three different columns distr_code, distr_de, and distr_it represent the information about the the district the municipality belongs to. Each of them individually fully functionally determines the other two of them. These three could be extracted into a separate relation with a primary key of distr_code for the relational schema.

3.4 Inclusion Dependency Discovery

The BINDER algorithm implementation in Metanome has been used for inclusion dependency discovery in this project. The algorithm used by this implementation is following the divide and conquer-based approach presented in [9]. The analysis has been run once over all five tables used in this project. Multi-column dependency discovery has initially been enabled, however, because it generates a very large number of 1024 different inclusion dependencies it became infeasible to analyze them. Therefore, the final evaluation uses the results of single-column

Determinant	Dependants
{}	status_gzp
{}	desc_i
{}	desc_d
{ogc_fid}	*
{gml_id}	*
{objectid}	*
{geom}	y_label, x_label, istat_code, name_i, name_d, gem_id
{geom, ...}	...
{x_label, ...}	...
{y_label, ...}	...
{istat_code, processo}	id_process
{istat_code, prozess}	id_process
{name_i, processo}	id_process
{name_i, prozess}	id_process
{name_d, processo}	id_process
{name_d, prozess}	id_process
{gem_id, processo}	id_process
{gem_id, prozess}	id_process
{processo}	prozess
{prozess}	processo
{istat_code}	name_i, name_d, gem_id
{name_d}	istat_code, name_i, gem_id
{name_i}	istat_code, name_d, gem_id
{code}	pericolo, gefahr
{pericolo}	code, gefahr
{gefahr}	code, pericolo
{gradodistudio}	bearbeitungstiefe, id_gs
{bearbeitungstiefe}	gradodistudio, id_gs

Table 8: Functional dependencies discovered in bz_hazard_landslide table using the Metanome HyFD algorithm. Multiple columns in the dependents means that all of them are functionally dependent on the determinant. Unique column combinations have been marked with * as dependents for brevity. Further, some functional dependencies that are not particularly insightful have been abbreviated using

inclusion dependency detection. The discovered multi-column inclusion dependencies have also been analyzed, but they largely do not contain any particularly relevant observations. This analysis can be used to find candidate for foreign keys between the different tables in the dataset which can be applied to the relational schema designed in Section 3.5.

Table 11 shows all the single-column inclusion dependencies discovered between the tables in this project. Among them are a number of coincidental inclusion dependencies between different dense identifiers that are purely valid because one of the ranges is smaller than the others. For example, all the inclusion dependencies involving the different ogc_fid columns belong to this category. One notable inclusion dependency that has been discovered, and which will become a foreign key in the relational schema, is that from all the istat_code attributes in the infrastructure and hazard tables to that same column in the municipalities tables. The columns nda_text_it and nda_text_de contain the same set of values in both of the infrastructure tables. Similarly, the columns id_gp, gem_id, name_d, name_i, desc_d, desc_i, bearbeitungstiefe, and gradodistudio each contain the same set of values between the two hazard tables. Interestingly, the analysis suggests that the set of municipality name in the infrastructure and hazard tables, such as those in the gem_text_d, gem_text_i, name_i, and name_d columns, are not exactly the same as the ones that are stored in the municipality table.

Looking at the multi-column inclusion dependencies, they contain many of the expected dependencies. For example both the inclusion dependency $\text{avalanche}[\text{istat_code}, \text{gem_id}, \text{name_i}, \text{desc_d}, \text{name_d}, \text{desc_i}, \text{status_gzp}] \subseteq \text{landslide}[\text{istat_code}, \text{gem_id}, \text{name_i}, \text{desc_d}, \text{name_d}, \text{desc_i}, \text{status_gzp}]$ and its reverse $\text{landslide}[\text{istat_code}, \text{gem_id}, \text{status_gzp}, \text{desc_d}, \text{desc_i}, \text{name_i}, \text{name_d}] \subseteq \text{avalanche}[\text{istat_code}, \text{gem_id}, \text{status_gzp}, \text{desc_d}, \text{desc_i}, \text{name_i}, \text{name_d}]$ are among them. Unfortunately, because a total of 473 multi-column inclusion dependencies were discovered by the algorithm, it is not feasible to extensively analyze them for further

Determinant	Dependants
{}	status_gzp
{}	desc_i
{}	desc_d
{}	id_phen
{ogc_fid}	*
{gml_id}	*
{objectid}	*
{geom}	x_label, y_label, istat_code, ..., id_gp, id_process, ..., id_gs, ...
{x_label, ...}	...
{y_label, ...}	...
{istat_code}	name_i, name_d, gem_id
{name_i}	istat_code, name_d, gem_id
{name_d}	istat_code, name_i, gem_id
{id_process}	processo, prozess
{processo}	id_process, prozess
{prozess}	id_process, processo
{code}	pericolo, gefahr
{pericolo}	code, gefahr
{gefahr}	code, pericolo
{id_gs}	gradodistudio, bearbeitungstiefe
{gradodistudio}	id_gs, bearbeitungstiefe
{bearbeitungstiefe}	id_gs, gradodistudio

Table 9: Functional dependencies discovered in bz_hazard_avalanche table using the Metanome HyFD algorithm. Multiple columns in the dependents means that all of them are functionally dependent on the determinant. Unique column combinations have been marked with * as dependents for brevity. Further, some functional dependencies that are not particularly insightful have been abbreviated using

Determinant	Dependants
{ogc_fid}	*
{gml_id}	*
{id}	*
{code}	*
{istat_code}	*
{belfiore_code}	*
{name_it}	*
{name_de}	*
{area}	*
{map_label}	*
{geom}	*
{health_district}	health_region
{distr_code}	distr_de, distr_it
{distr_de}	distr_code, distr_it
{distr_it}	distr_code, distr_de

Table 10: Functional dependencies discovered in bz_admin_municipality table using the Metanome HyFD algorithm. Multiple columns in the dependents means that all of them are functionally dependent on the determinant. Unique column combinations have been marked with * as dependents for brevity.

insights.

Dependant	Referenced
node.codealt	*
node.blp	*
node.ogc_fid	line.ogc_fid, landslide.ogc_fid, avalanche.ogc_fid
node.istat_code	municipality.istat_code
node.nda_text_it	line.nda_text_it
node.nda_text_de	line.nda_text_de
line.codealt	*
line.blp	*
line.istat_code	municipality.istat_code
line.nda_text_it	node.nda_text_it
line.nda_text_de	node.nda_text_de
landslide.ogc_fid	line.ogc_fid
landslide.id_gs	avalanche.id_gs
landslide.id_gp	avalanche.id_gp
landslide.gem_id	avalanche.gem_id
landslide.name_d	avalanche.name_d
landslide.name_i	avalanche.name_i
landslide.desc_d	avalanche.desc_d
landslide.desc_i	avalanche.desc_i
landslide.bearbeitungstiefe	avalanche.bearbeitungstiefe
landslide.gradodistudio	avalanche.gradodistudio
landslide.istat_code	avalanche.istat_code, municipality.istat_code
landslide.status_gzp	municipality.ogc_fid, line.ogc_fid, landslide.ogc_fid, landslide.id_gp, avalanche.ogc_fid, avalanche.status_gzp, avalanche.id_gp, node.ogc_fid
avalanche.ogc_fid	line.ogc_fid, landslide.ogc_fid
avalanche.id_gp	landslide.id_gp
avalanche.gem_id	landslide.gem_id
avalanche.name_d	landslide.name_d
avalanche.name_i	landslide.name_i
avalanche.desc_d	landslide.desc_d
avalanche.desc_i	landslide.desc_i
avalanche.bearbeitungstiefe	landslide.bearbeitungstiefe
avalanche.gradodistudio	landslide.gradodistudio
avalanche.id_phen	*
avalanche.istat_code	landslide.istat_code, municipality.istat_code
avalanche.status_gzp	municipality.ogc_fid, line.ogc_fid, node.ogc_fid, landslide.status_gzp, landslide.id_gp, avalanche.ogc_fid, avalanche.id_gp, landslide.ogc_fid
municipality.ogc_fid	node.ogc_fid, line.ogc_fid, landslide.ogc_fid, avalanche.ogc_fid
municipality.health_region	line.shape_len, landslide.gem_id, avalanche.gem_id, municipality.id, municipality.code, municipality.distr_code
municipality.distr_code	line.shape_len, landslide.gem_id, avalanche.gem_id, municipality.id, municipality.code
municipality.health_district	municipality.id, municipality.code

Table 11: All single-column inclusion dependencies discovered across all tables using the Metanome BINDER algorithm. Multiple columns in the referenced column means that all of them include the dependant. Trivial inclusion dependencies where the dependant is empty have been marked with * for brevity.

3.5 Relational Database Schema

Based on the above data profiling results, a relational schema has been developed that captures all the relevant data for answering the application questions outlined in the introduction. To exploit metadata extracted from dependency discovery algorithms, the schema design relies heavily on removing data redundancies highlighted by the functional dependency analyses. Further, the unique column combination and inclusion dependency profiling results dictate the schema’s relational constraints such as the primary and foreign keys. Some attributes present

in the raw dataset have also been excluded from the developed schema. The exclusion is guided by two main criteria, data quality and redundancy findings from the data profiling steps and the relevance to the application domain outlined in Section 1.

InfrastructureType		
Attribute	Data Type	Constraints
<u>code</u>	INTEGER	PK
name_it	VARCHAR[64]	UNIQUE
name_de	VARCHAR[64]	UNIQUE

HazardProcess		
Attribute	Data Type	Constraints
<u>code</u>	CHAR[2]	PK
name_it	VARCHAR[64]	UNIQUE
name_de	VARCHAR[64]	UNIQUE

InfrastructureLine		
Attribute	Data Type	Constraints
<u>code</u>	INTEGER	PK
type_code	INTEGER	FK: InfrastructureType
istat_code	INTEGER	OPT, FK: Municipality
geom	GEOMETRY	

InfrastructureNode		
Attribute	Data Type	Constraints
<u>code</u>	INTEGER	PK
type_code	INTEGER	FK: InfrastructureType
istat_code	INTEGER	FK: Municipality
geom	GEOMETRY	

DangerLevel		
Attribute	Data Type	Constraints
<u>code</u>	INTEGER	PK
name_it	VARCHAR[64]	UNIQUE
name_de	VARCHAR[64]	UNIQUE

District		
Attribute	Data Type	Constraints
<u>code</u>	INTEGER	PK
name_it	VARCHAR[64]	UNIQUE
name_de	VARCHAR[64]	UNIQUE

LandslideHazard		
Attribute	Data Type	Constraints
<u>code</u>	INTEGER	PK
danger_code	INTEGER	FK: DangerLevel
process_code	CHAR[2]	FK: HazardProcess
istat_code	INTEGER	FK: Municipality
geom	GEOMETRY	

AvalancheHazard		
Attribute	Data Type	Constraints
<u>code</u>	INTEGER	PK
danger_code	INTEGER	FK: DangerLevel
process_code	CHAR[2]	FK: HazardProcess
istat_code	INTEGER	FK: Municipality
geom	GEOMETRY	

Municipality		
Attribute	Data Type	Constraints
<u>istat_code</u>	INTEGER	PK
name_it	VARCHAR[64]	UNIQUE
name_de	VARCHAR[64]	UNIQUE
name_ld	VARCHAR[64]	OPT
zip_code	INTEGER	
distr_code	INTEGER	FK: District
geom	GEOMETRY	

Table 12: Specification of the relational schema designed for the data source in this project. Primary keys are underlined and marked with "PK", while foreign keys are marked with "FK: Relation". Foreign keys always refer to the primary key of the target relation. Optional attributes are marked with "OPT". Attributes not marked as such have a not-null constraint by default.

The complete specification of the developed relational schema is shown in Table 12. Note that, as will be explained below, some attributes have been renamed with respect to the original dataset for better understanding. For example, most primary key identifiers have been named code. The rest of this section will describe in detail the schema and the rationale for each design depiction taken, referencing both the relevance to the application domain the results of data profiling.

Schema Normalization and Entity Extraction Based on the functional dependencies discovered in Section 3.3, a number of entities have been extracted from the raw table to remove data redundancy.

Within bz_admin_municipality, the district related attributes distr_code, distr_de, and distr_it each mutually determine one another. This means they can be isolated into a separate District relation to prevent duplication across municipalities. In this relation the distr_code, distr_de, and distr_it attributes are named respectively code, name_de, and name_it. This removes the redundant use "distr" and makes it clear that the latter two are the name.

For both the `bz_infrastructure_line` and `bz_infrastructure_node` the code, which identifies the type of infrastructure, functionally determine the German and Italian names in `bez_d` and `bez_i`. For this reason, these attributes were extracted into an `InfrastructureType` relation. Note that for simplicity, and because data profiling has shown that the two have disjoint code domain, the types in both source tables were extracted into a single relation. Note that `bez_d` and `bez_i` were renamed respectively to `name_de` and `name_it` for consistency with other relations.

In the hazard datasets `bz_hazard_landslide` and `bz_hazard_avalanche` the danger classifications is composed of `code`, `pericolo`, and `gefahr`, and these attributes fully determine each other, therefore suggesting the creation of a distinct `DangerLevel` table. In this extracted entity, the `pericolo` and `gefahr` attributes are instead names `name_it` and `name_de` respectively. Note that while both hazard kinds use the same danger classification scale, the codes are different and also the names differ very slightly. For this reason, a small data transformation is applied to transform them to have the same names and identifiers.

Finally, similar to the danger level, also the hazard process is formed by the three attributes `id_process`, `processo`, and `prozess` in both hazard datasets. Note that unlike with the danger level, data profiling has shown that the hazard process uses the same space of identifiers in both original tables, and they also share the same names for each process. For this reason they can trivially use the same extracted entity. In the extracted entity the attributes' `id_process`, `processo`, and `prozess` are named `code`, `name_it`, and `name_de` for consistency.

Primary and Foreign Key Definitions The results of the unique column combination and inclusion dependency discovery results have been used to determine the chosen primary key and foreign key constraints. The profiling results have also been used to motivate a number of non-primary key constraints.

The inclusion dependency analysis confirmed that `istat_code` from the infrastructure and hazard datasets is fully included within the `istat_code` column of the administrative municipality dataset. Consequently, even though the unique column combination discovery identified also numerous other possible primary keys, `istat_code` has been selected as primary key for the municipality table and serves also as a foreign key in all the infrastructure and hazard tables.

For both of the infrastructure tables, the unique column combination discovery identified `ogc_fid`, `gml_id`, and `objectid` as minimal unique combinations. The identifier in `objectid` has been chosen as the primary key because it is a sparse integer that is unique even across the different tables and is an appropriate primary key in the absences of other information. Note that in both `InfrastructureLine` and `InfrastructureNode` this identifier has been renamed to `code` for consistency with the other tables.

Finally, also for the hazard datasets, the unique column combination results present the same three candidate keys `ogc_fid`, `gml_id`, and `objectid`. For this reason, exactly the same choice has been made. `objectid` is used as the primary key of both the `LandslideHazard` and `AvalancheHazard` relations, but like with the infrastructure tables, these have been renamed to `code`.

Attributes Excluded from the Relational Schema As already mentioned, some columns of the original datasets have not been incorporated into the relational schema, either for reasons of data quality, redundancy, or insufficient relevance to the chosen application domain.

First, data profiling has shown that several columns contain no usable information, making their inclusion unnecessary for schema design. `codealt` and `blp` in the `bz_infrastructure_line` and `bz_infrastructure_node` tables have been confirmed by profiling to both contain 100% missing values across all 47551 line records and 4223 node records. Similarly, the value for `id_phen` in the `bz_hazard_avalanche` table is missing for all 13071 rows. Further, while the `status_gzp`, `desc_i`, and `desc_d` always contain a value, this value is constant across every row. Additionally, these indicate administrative approval status and therefore offer no utility for exposure queries.

Some other attributes have been omitted in order to avoid redundancy. This is the case for those attributes that have been extracted into separate entities like the municipality names in the hazard and infrastructure datasets. Storing municipality names inside the infrastructure and hazard tables would violate Third Normal Form. Instead, storing `istat_code` as a foreign key to the Municipality relation preserves this information cleanly. In addition to the attributes made redundant by normalization, all the datasets also contain multiple different identifiers, such as `ogc_fid`, `gml_id` and `objectid` for the infrastructure tables. Having multiple of these identifiers, which are otherwise not used and have no inherent semantic values, does not give any additional benefit. For this reason, only a single identifier has been selected as the primary key for each relation. All other identifiers present in these tables have been dropped from the relational schema. Furthermore, also the attributes `shape_len` and `area`, which have been shown to be derivable directly from the geometry, have been excluded from the relational schema to reduce redundancy. Instead, these properties can be computed from the geometry as part of the SQL query during mapping or after the fact in the application.

Several attributes in the original datasets exist solely for rendering or front-end visualization, adding no

semantic value to database queries. The point coordinates `x_label` and `y_label` in the hazard tables are purely for placing map text labels in software and are therefore dropped from the relational schema. The `nda_text_*` columns contain the static UI button labels "Öffne" and "Apri" for web portals, while `nda_link_*` contains external URLs to land-use regulations. Neither contributes to answering risk assessment queries and are therefore excluded. `map_label` has also been dropped from the Municipality relation, because it contains only a pre-formatted display string concatenating bilingual municipality names, which can be constructed dynamically in SPARQL or SQL when needed.

Finally, in addition to these front-end oriented attributes, there are also some attributes that were excluded because they are not relevant to the chosen application domain. In the municipality dataset, `health_region`, and `health_district` contain healthcare administrative boundaries. While these could be interesting for infrastructure risk assessments in some cases, because they only contain identifiers and no actual labels, it is difficult to use them without additional datasets, and they were therefore omitted. Further, the columns `id_gp`, `gradodistudio`, `bearbeitungstiefe`, and `id_gs` in the hazards datasets contain internal codes tracking survey depth. While relevant to geological surveying, they have been dropped from the relational schema because they are not relevant to the chosen application.

Proposed Data Types The data types for the different attributes in the relational schema have been chosen mostly based on what has been discovered during the single column analysis performed in Section 3.1. For example, the type `VARCHAR[64]` has been used for all fields that contain small string, such as for the name fields. This size has been determined during data profiling to be large enough for all names in the dataset. Note that during data profiling it was discovered that some columns, while they are formatted as real numbers, they actually only contain whole integer values. This is primarily the case for the different identifier columns such as for example the `istat_code` or `distr_code` in the municipality dataset. For these columns, a minor data transformation has been applied to transform them into proper integer variables and given `INTEGER` type in the schema. The only exception to the use of integers for the identifiers is that the HazardProcess makes use of a two character code. Appropriately, a data type of `CHAR[2]` is used for this attribute. Finally, all geometry information will be stored in a PostGIS [12] instance using the `GEOMETRY` data type.

4 Data Integration

This section of the report discusses the data integration part of the project. Section 4.1 describes the designed OWL 2 QL ontology and explains the design decisions taken. Section 4.2 covers the implemented mapping from the relational schema to the developed ontology. Finally, Section 4.3 gives an overview of the functionality that has been implemented in the developed application.

4.1 Ontology

An OWL 2 QL [6] ontology that contains relevant classes and properties has been developed as part of the project. Figure 4 depicts the ontology using the graphical notation introduced in the course [1]. The same ontology has also been represented in Protégé [7] for this project.

In order to fulfill the project's goal of assessing infrastructure vulnerability, the ontology models the intersection of critical infrastructure and natural hazard zones. It is designed in compliance with the OWL 2 QL profile that we learned in the course to guarantee efficient query answering via the Ontop virtual knowledge graph system. The conceptual schema is divided into four main domains: spatial entities, infrastructure assets, hazard areas, and administrative boundaries.

4.1.1 Class Hierarchy and Disjointness

At the root of the spatial domain is the `GeometricObject` class, which serves as a superclass for `Infrastructure`, `HazardZone`, and `Municipality`. This allows shared spatial properties to be abstracted at a high level, instead of duplicating them for each of the subclasses. Even though they are part of the same class hierarchy, the classes `Infrastructure`, `HazardZone`, and `Municipality` are defined to be mutually disjoint.

The `Infrastructure` class is subclassed along two orthogonal dimensions. The infrastructure is first divided based on their structure into `InfrastructureNode` and `InfrastructureLine`. These two are declared as mutually disjoint, since an infrastructure asset cannot simultaneously be a point and a line. Then the infrastructure objects are also categorized thematically into `CommunicationInfrastructure`, `EnergyInfrastructure`, `WaterInfrastructure`, and `WasteInfrastructure`. These thematic classes are also strictly disjoint, ensuring that a single infrastructure element cannot simultaneously belong to distinct functional networks. Note that we also add `ElectricityInfrastructure` as

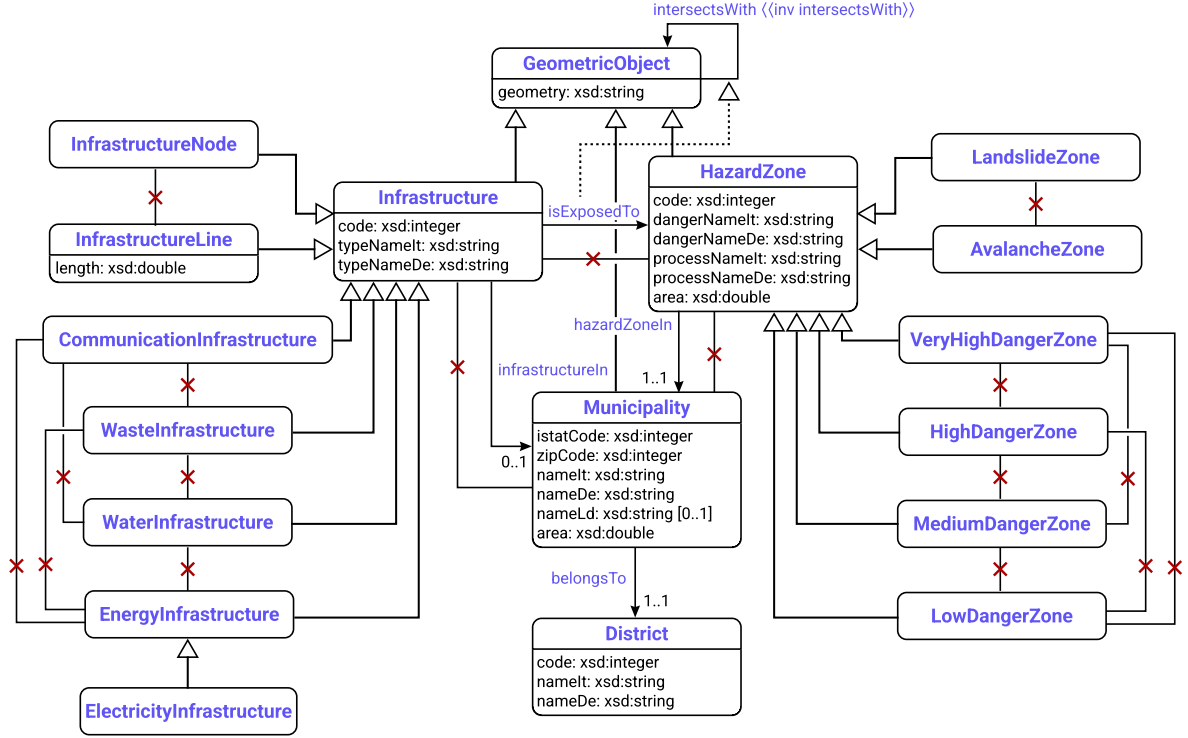


Figure 4: The OWL 2 QL ontology that has been developed for the project. The diagram uses the graphical notation introduced in the course.

an additional subclass of EnergyInfrastructure. Note that while most of the infrastructure types can be clearly split into one of them, some infrastructure elements may not belong to any of these thematic subclasses.

Similarly, the HazardZone class is also specialized along two dimensions. First, it is split into disjoint LandslideZone and AvalancheZone classes. To capture the severity of the hazard's danger, as identified in the pericolo and Gefahr columns during the data profiling phase, HazardZone is further classified into the four mutually disjoint danger levels VeryHighDangerZone, HighDangerZone, MediumDangerZone, and LowDangerZone.

Finally, the administrative domain consists of the Municipality and District classes, which provide the information required to for example aggregate the exposed assets per municipality or district.

4.1.2 Object Properties

The relationships between the above described classes are formalized through specific object properties designed to answer the project's core application queries. General spatial overlaps are handled by the intersectsWith property defined between two GeometricObject. Note that this object property is explicitly defined to be symmetric, which can be represented in OWL 2 QL by a object property that is its own inverse. To explicitly address vulnerability of infrastructure to hazards, isExposedTo is introduced as a sub-property of intersectsWith, linking Infrastructure directly to HazardZone. These object properties will later be populated by performing spatial joins over the geometries of the respective elements. Note that there are no cardinality restrictions on these properties, since a single geometry can intersect with an arbitrary number of other geometries.

Administrative containment is modeled using the infrastructureIn and hazardZoneIn properties, which link physical and hazard entities to a Municipality. Finally, the belongsTo property maps a Municipality to a District. Note that these properties are envisioned to be derived from the municipality specified in the infrastructure and hazard datasets instead of using geometric intersection. For this reason, the properties map each Infrastructure element to at most one Municipality, and each HazardZone to exactly one Municipality. The infrastructureIn property does not use mandatory participation, because data profiling has shown that some infrastructure lines are missing municipality information. Further, mandatory participation and functionality constraints on the belongsTo object property enforce that each Municipality belonging to exactly one District. This enforces the structural integrity observed in the relational datasets. While functionality of object or data properties can not be enforced in OWL 2 QL, these are still included in Fig. 4 for completeness.

4.1.3 Data Properties

Many of the data properties are directly mapped from the insights extracted during the elementary data analysis during relational schema creation. For example, `istatCode`, `zipCode`, `area`, and localized names `nameIt`, `nameDe`, and `nameLd` are assigned to `Municipality` and directly taken from the associated relation. Reflecting the high number of missing values discovered in the single-column analysis of the `bz_admin_municipality` table, the Ladin name property `nameLd` is explicitly constrained with a cardinality of `[0...1]` to indicate that it is optional. The data type `xsd:integer` is used for `istatCode` and `zipCode`, while `xsd:string` is used for all the localized names. Lastly, `xsd:double` is used as the data type for the `area` data property. Similarly, the `District` class exposes the district code using type `xsd:integer` as well as the localized names in `nameIt` and `nameDe`, both using the data type `xsd:string`.

For the infrastructure domain, the `Infrastructure` class contains data properties associating in addition to the code also the localized names for the infrastructure type in `typeNameIt` and `typeNameDe`. While in the relational schema these were in a separate entity, the ontology instead models them as data properties on the `Infrastructure` class. In addition to these general data properties present for all infrastructure elements, there is also the length data property that exists only for the `InfrastructureLine` class.

In the `HazardZone` class we also expose the code as a data property with type `xsd:integer`. Even though there are already separate subclasses for each of the four possible danger levels of the hazard zone, the `HazardZone` class still has data properties `dangerNameIt` and `dangerNameDe` in order to be able to display the appropriate names in the application output. Further, `HazardZone` also contains localized names for the type of avalanche or landslide process by means of the data properties `processNameIt` and `processNameDe`. Again, these are represented by a separate relation in the relational schema but for the ontology they have been converted to data properties. Additionally, the `HazardZone` also contains an `area` data property.

Finally, spatial geometries are uniformly captured across all physical entities via the `geometry` property on the `GeometricObject` class. The property is stored as strings, of type `xsd:string`, representing the Well-Known Text format discovered during profiling.

4.2 Virtual Knowledge Graph Mappings

The virtual knowledge graph mappings from the relational schema defined in Section 3.5 to the ontology covered in Section 4.1 has been performed using the Ontop plugin for Protégé [2]. This section lists and describes a representative subset of the developed mappings.

4.2.1 IRI Generation

The Ontop virtual knowledge graph fundamentally relies on generating unique, persistent Internationalized Resource Identifiers (IRIs) for every entity in the relational database. This is achieved using URI templates defined in the target clauses of the mappings, which bind primary keys from the relational schema to the namespace of the ontology. The templates `:district/{code}`, `:municipality/{istat_code}`, `:infrastructure-line/{code}`, `:infrastructure-node/{code}`, `:hazard-landslide/{code}`, and `:hazard-avalanche/{code}` are used in the developer mappings for the districts, municipalities, infrastructure lines, infrastructure node, landslide zones, and avalanche zones respectively. This guarantee that each physical or administrative object is represented as a distinct web resource. Note that both the infrastructure line and nodes as well as hazard zones are all collected into the different IRI templates. This is not strictly necessary, because as has been discovered during data profiling, the primary key space for the two tables is disjoint. However, making them use different IRI templates make the intention clearer and could possibly be used for some optimization. By consistently reusing these templates across all the mapping declarations, the virtual knowledge graph seamlessly merges data from disparate relational queries into a unified graph, allowing properties and classifications to be appended to the same entity incrementally.

4.2.2 Simple Mappings

The basic structural mappings directly instantiate the primary classes of the ontology and populate their basic data and object properties using straightforward SQL `SELECT` statements.

The below mappings `MAP-DISTRICT` and `MAP-MUNICIPALITY` instantiate the municipality and district elements. Aside from asserting the basic class membership, they also map scalar data properties such as the ISTAT code, area, and different localized string literals. Crucially, the mapping for the municipality leverages the existing foreign key `distr_code` to assert the `:belongsTo` object property, linking the municipality directly to its parent district's IRI. Further, the mapping utilizes the spatial function `ST_AsText(geom)` in the source SQL query to cast the PostGIS binary geometries into standard Well-Known Text strings and binds the result to the `:geometry` data

property. The same pattern is used for all geometry carrying classes. Additionally, for municipalities we use the spatial function `ST_Area(ST_MakeValid(geom)::geography)` to compute the real world size of the represented geometry.

MAP-DISTRICT

Source: SQL Query <pre>SELECT code, name_it, name_de FROM District</pre>	Target: Triples Template <pre>:district/{code} a :District ; :districtCode {code}^^xsd:integer ; :districtNameIt {name_it}^^xsd:string ; :districtNameDe {name_de}^^xsd:string .</pre>
---	---

MAP-MUNICIPALITY

Source: SQL Query <pre>SELECT istat_code, zip_code, distr_code, name_it, name_de, name_ld, ST_Area(ST_MakeValid(geom)::geography) AS area ST_AsText(geom) AS geom FROM Municipality</pre>	Target: Triples Template <pre>:municipality/{istat_code} a :Municipality ; :istatCode {istat_code}^^xsd:integer ; :zipCode {zip_code}^^xsd:integer ; :nameIt {name_it}^^xsd:string ; :nameDe {name_de}^^xsd:string ; :nameLd {name_ld}^^xsd:string ; :area {area}^^xsd:double ; :geometry {geom}^^xsd:string ; :belongsTo :district/{distr_code} .</pre>
--	---

The mapping MAP-INFRASTRUCTURE-LINE populates the `:InfrastructureLine` class. The structure is very similar to that of the municipality mapping, mapping the `:geometry` data property to the geometry and also asserting the topological `:infrastructureIn` property using the `istat_code` foreign key. Further, we also compute the length of the infrastructure line. Nearly the identical mapping, except for the omission of the `:lineLength` data property and use of `:InfrastructureNode` class, has also been created for the infrastructure nodes and is shown in MAP-INFRASTRUCTURE-NODE.

MAP-INFRASTRUCTURE-LINE

Source: SQL Query <pre>SELECT code, istat_code, ST_Length(geom::geography) AS len, ST_AsText(geom) AS geom FROM InfrastructureLine</pre>	Target: Triples Template <pre>:infrastructure-line/{code} a :Infrastructureline ; :infrastructureIn :municipality/{istat_code} ; :infrastructureCode {code}^^xsd:integer ; :lineLength {len}^^xsd:double ; :geometry {geom}^^xsd:string .</pre>
---	--

MAP-INFRASTRUCTURE-NODE

Source: SQL Query <pre>SELECT code, istat_code, ST_AsText(geom) AS geom FROM InfrastructureNode</pre>	Target: Triples Template <pre>:infrastructure-node/{code} a :InfrastructureNode ; :infrastructureIn :municipality/{istat_code} ; :infrastructureCode {code}^^xsd:integer ; :geometry {geom}^^xsd:string .</pre>
---	--

Finally, also the hazard zones get mappings similar to those for the infrastructure assets. MAP-HAZARD-LANDSLIDE below shows the mapping for the landslide hazard zones. Similar to the infrastructure mappings, these instantiate the core hazard zones, bind their identifier and geometries, and establish their spatial relationship to municipalities via the `:hazardZoneIn` object property. The mapping for the avalanche hazard zones MAP-HAZARD-AVALANCHE is nearly identical but using `AvalancheHazard` as the source table and `:LandslideZone` as the target class. Like with the municipalities, also here we compute the area of the covered area based on the underlying geometry.

MAP-HAZARD-LANDSLIDE

Source: SQL Query <pre>SELECT code, istat_code, ST_Area(ST_MakeValid(geom)::geography) AS area ST_AsText(geom) AS geom FROM LandslideHazard</pre>	Target: Triples Template <pre>:hazard-landslide/{code} a :LandslideZone ; :hazardZoneIn :municipality/{istat_code} ; :hazardCode {code}^^xsd:integer ; :hazardArea {area}^^xsd:double ; :geometry {geom}^^xsd:string .</pre>
---	---

MAP-HAZARD-AVALANCHE

Source: SQL Query <pre>SELECT code, istat_code, ST_Area(ST_MakeValid(geom)::geography) AS area ST_AsText(geom) AS geom FROM AvalancheHazard</pre>	Target: Triples Template <pre>:hazard-avalanche/{code} a :AvalancheZone ; :hazardZoneIn :municipality/{istat_code} ; :hazardCode {code}^^xsd:integer ; :hazardArea {area}^^xsd:double ; :geometry {geom}^^xsd:string .</pre>
---	---

4.2.3 Properties Requiring Relational Joins

Because the elementary data analysis justified normalizing lookup tables to remove redundancies, the virtual knowledge graph mappings must bridge these normalized structures using SQL joins to construct some of the data properties that in the ontology have been assigned directly to the entities.

The MAP-INFRASTRUCTURE-LINE-TYPE and MAP-INFRASTRUCTURE-NODE-TYPE mappings use an inner join to retrieve the German and Italian descriptions from the normalized InfrastructureType table and insert them into the knowledge graph as :typeNameDe and :typeNameIt data properties to the existing :infrastructure-line/{code} and :infrastructure-node/{code} IRIs.

MAP-INFRASTRUCTURE-LINE-TYPE

Source: SQL Query	Target: Triples Template
<pre>SELECT InfrastructureLine.code, name_it, name_de FROM InfrastructureLine, InfrastructureType WHERE type_code = InfrastructureType.code</pre>	<pre>:infrastructure-line/{code} :typeNameIt {name_it}^^xsd:string ; :typeNameDe {name_de}^^xsd:string .</pre>

MAP-INFRASTRUCTURE-NODE-TYPE

Source: SQL Query	Target: Triples Template
<pre>SELECT InfrastructureNode.code, name_it, name_de FROM InfrastructureNode, InfrastructureType WHERE type_code = InfrastructureType.code</pre>	<pre>:infrastructure-node/{code} :typeNameIt {name_it}^^xsd:string ; :typeNameDe {name_de}^^xsd:string .</pre>

The same kind of mappings are used for mapping the hazard process and danger level names by execute similar joins against the DangerLevel and HazardProcess tables. Below these mappings for the landslide hazard, MAP-HAZARD-LANDSLIDE-DANGER and MAP-HAZARD-LANDSLIDE-PROCESS, are shown, however, the mappings for the avalanche hazard are largely the same except for the change in IRI and class. This allows the danger and process descriptions to be attached directly to the hazard zone instances, shielding the ontology consumer from the underlying relational schema's normalization.

MAP-HAZARD-LANDSLIDE-DANGER

Source: SQL Query	Target: Triples Template
<pre>SELECT LandslideHazard.code, name_it, name_de FROM LandslideHazard, DangerLevel WHERE danger_code = DangerLevel.code</pre>	<pre>:hazard-landslide/{code} :dangerNameIt {name_it}^^xsd:string ; :dangerNameDe {name_de}^^xsd:string .</pre>

MAP-HAZARD-LANDSLIDE-PROCESS

Source: SQL Query	Target: Triples Template
<pre>SELECT LandslideHazard.code, name_it, name_de FROM LandslideHazard, HazardProcess WHERE process_code = HazardProcess.code</pre>	<pre>:hazard-landslide/{code} :processNameIt {name_it}^^xsd:string ; :processNameDe {name_de}^^xsd:string .</pre>

4.2.4 Semantic Sub-classification

To align the raw data with the ontology's orthogonal classification schemes for the infrastructure assets and hazard zones, some mappings are required instantiate semantic subclasses based on specific textual values found in the relational database.

For the infrastructure domain classification, these mappings use SQL IN clauses with a set of literals in the WHERE condition to group specific textual descriptions, for example "Methangasleitung" and "Umspannstation") into their respective functional ontology classes. Four different mappings exists for both the infrastructure lines and nodes, one for each of the ontology classes :EnergyInfrastructure, :ElectricityInfrastructure, :CommunicationInfrastructure, :WaterInfrastructure, and :WasteInfrastructure. This dynamically asserts multiple inheritance, as an entity will be typed as both for example an :InfrastructureLine and an :EnergyInfrastructure. The mappings MAP-INFRASTRUCTURE-LINE-ENERGY and MAP-INFRASTRUCTURE-NODE-WATER show two representative examples for this kind of mapping.

A similar approach is taken for mapping of the hazard severity classification. Like for the infrastructure domain, these mappings translate the specific German string identifiers for danger levels into the defined ontology subclasses. This value-driven instantiation enables complex semantic queries based on danger severity without requiring users to know the exact string values used in the source database. Unlike the above mappings, these use direct equality because the mapping from string identifiers to ontology classes is one-to-one in this case. Only MAP-HAZARD-LANDSLIDE-VERYHIGH and MAP-HAZARD-AVALANCHE-MEDIUM are shown below, however, for all remaining combinations of danger level and hazard type nearly identical mappings have been created.

MAP-INFRASTRUCTURE-LINE-ENERGY

Source: SQL Query

```
SELECT InfrastructureLine.code
FROM InfrastructureLine, InfrastructureType
WHERE type_code = InfrastructureType.code
AND name_de IN (
'Methangasleitung', 'Hochspannungsleitung',
'Mittelspannungsleitung', 'Fernwärmeleitung',
'Methangasleitung_Bannstreifen',
'Hochspannungsleitung_Bannstreifen')
```

Target: Triples Template

```
:infrastructure-line/{code}
a :EnergyInfrastructure .
```

MAP-INFRASTRUCTURE-NODE-WATER

Source: SQL Query

```
SELECT InfrastructureNode.code
FROM InfrastructureNode, InfrastructureType
WHERE type_code = InfrastructureType.code
AND name_de IN (
'Wasserfassung', 'Reservoir', 'Pumpstation')
```

Target: Triples Template

```
:infrastructure-node/{code}
a :WaterInfrastructure .
```

MAP-HAZARD-LANDSLIDE-VERYHIGH

Source: SQL Query

```
SELECT LandslideHazard.code
FROM LandslideHazard, DangerLevel
WHERE danger_code = DangerLevel.code
AND name_de = 'Sehr_hohe_Gefahrenstufe(H4)'
```

Target: Triples Template

```
:hazard-landslide/{code} a :VeryHighDangerZone .
```

MAP-HAZARD-AVALANCHE-MEDIUM

Source: SQL Query

```
SELECT AvalancheHazard.code
FROM AvalancheHazard, DangerLevel
WHERE danger_code = DangerLevel.code
AND name_de = 'Mittlere_Gefahrenstufe(H2)'
```

Target: Triples Template

```
:hazard-avalanche/{code} a :MediumDangerZone .
```

4.2.5 Spatial Object Properties

The most computationally powerful aspect of the developed virtual knowledge graph is the dynamic generation of object properties using spatial database functions, eliminating the need to pre-compute and store complex topological relationships. In the ontology these are represented using the `:intersectsWith` object property defined between all geometric objects and its sub-property `:isExposedTo` which is explicitly defined between infrastructure and hazards. Note that the `:intersectsWith` object property has been defined as symmetric, and therefore only a single direction must be specified, with the other being inferred by the Ontop system. Given the 5 different tables, two for the infrastructure assets, two for the hazard zones, and one for the municipalities a total of 15 different mappings are necessary to capture all possible intersection relationships. These mappings utilize the PostGIS `ST_Intersects()` function directly within the SQL source query.

First, mappings have been created to populate the `:isExposedTo` relation. By performing a spatial join between the geometries of infrastructure assets and hazard zones, Ontop dynamically asserts the `:isExposedTo` object property whenever a physical overlap occurs. This directly solves the project's primary analytical goal of finding infrastructure assets vulnerable to different hazard. 4 different mappings of this type have been created, one for each combination of infrastructure and hazard kind. Below two of them, `MAP-ISEXPOSEDTO-LINE-LANDSLIDE` and `MAP-ISEXPOSEDTO-NODE-AVALANCHE`, are shown.

MAP-ISEXPOSEDTO-LINE-LANDSLIDE

Source: SQL Query

```
SELECT InfrastructureLine.code AS infra_code,
LandslideHazard.code AS hazard_code
FROM InfrastructureLine, LandslideHazard
WHERE ST_Intersects(
InfrastructureLine.geom, LandslideHazard.geom)
```

Target: Triples Template

```
:infrastructure-line/{infra_code}
:isExposedTo :hazard-landslide/{hazard_code} .
```

MAP-ISEXPOSEDTO-NODE-AVALANCHE

Source: SQL Query

```
SELECT InfrastructureNode.code AS infra_code,
AvalancheHazard.code AS hazard_code
FROM InfrastructureNode, AvalancheHazard
WHERE ST_Intersects(
InfrastructureNode.geom, AvalancheHazard.geom)
```

Target: Triples Template

```
:infrastructure-node/{infra_code}
:isExposedTo :hazard-avalanche/{hazard_code} .
```

Next, there are 4 mappings to connect the infrastructure and hazard zone entities to the municipalities using the `:intersectsWith` object property. While base mappings use foreign keys for `:infrastructureIn`, these mappings use `ST_Intersects()` to assert the broader `:intersectsWith` property based on physical boundaries, accommodating edge cases where an infrastructure line might physically cross municipal borders beyond its

primary assigned one. Two representative examples of these, MAP-INTERSECTSWITH-LINE-MUNICIPALITY and MAP-INTERSECTSWITH-AVALANCHE-MUNICIPALITY are shown below.

MAP-INTERSECTSWITH-LINE-MUNICIPALITY

Source: SQL Query

```
SELECT code, Municipality.istat_code
FROM InfrastructureLine, Municipality
WHERE ST_Intersects(
  InfrastructureLine.geom, Municipality.geom)
```

Target: Triples Template

```
:infrastructure-line/{code}
:intersectsWith :municipality/{istat_code} .
```

MAP-INTERSECTSWITH-AVALANCHE-MUNICIPALITY

Source: SQL Query

```
SELECT code, Municipality.istat_code
FROM AvalancheHazard, Municipality
WHERE ST_Intersects(
  AvalancheHazard.geom, Municipality.geom)
```

Target: Triples Template

```
:hazard-avalanche/{code}
:intersectsWith :municipality/{istat_code} .
```

Finally, the same framework using ST_Intersects() can be naturally extended to map relationships between similar geometries such as between two infrastructure assert, two municipalities, or two hazard zones. These extended topological intersections account for the remaining 7 possible ways intersecting between the 5 tables. By joining tables against themselves or their counterparts, the virtual knowledge graph can expose the complete topological network of the physical domain natively as object properties. The mappings MAP-INTERSECTSWITH-LINE-NODE and MAP-INTERSECTSWITH-MUNICIPALITY-MUNICIPALITY are two out of the 7 mappings of this form.

MAP-INTERSECTSWITH-LINE-NODE

Source: SQL Query

```
SELECT InfrastructureLine.code AS code1,
  InfrastructureNode.code AS code2
FROM InfrastructureLine, InfrastructureNode
WHERE ST_Intersects(
  InfrastructureLine.geom,
  InfrastructureNode.geom)
```

Target: Triples Template

```
:infrastructure-line/{code1}
:intersectsWith :infrastructure-node/{code2} .
```

MAP-INTERSECTSWITH-MUNICIPALITY-MUNICIPALITY

Source: SQL Query

```
SELECT T1.istat_code AS code1,
  T2.istat_code AS code2
FROM Municipality AS T1, Municipality AS T2
WHERE ST_Intersects(T1.geom, T2.geom)
```

Target: Triples Template

```
:municipality/{code1}
:intersectsWith :municipality/{code2} .
```

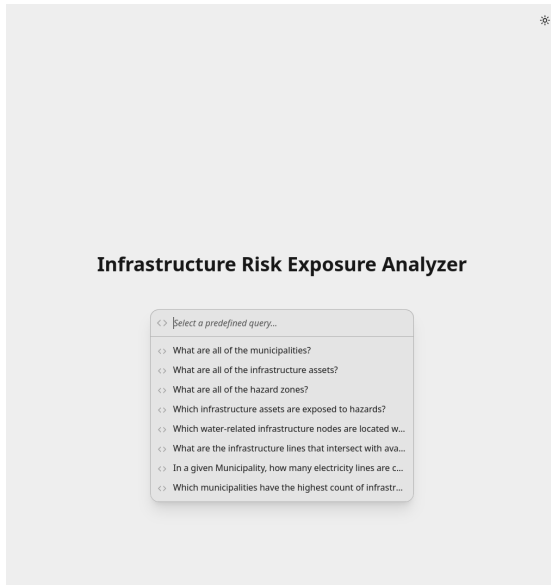
4.3 Application

The application for this project has been implemented as a custom single-page web application using Vite [14], React [5] and React Router [13]. It offers an interactive map and results table which dynamically update based on a number of different configurations options provided to the user. To obtain the results of the queries, the application connects to the running Ontop system via its SPARQL [3] endpoint.

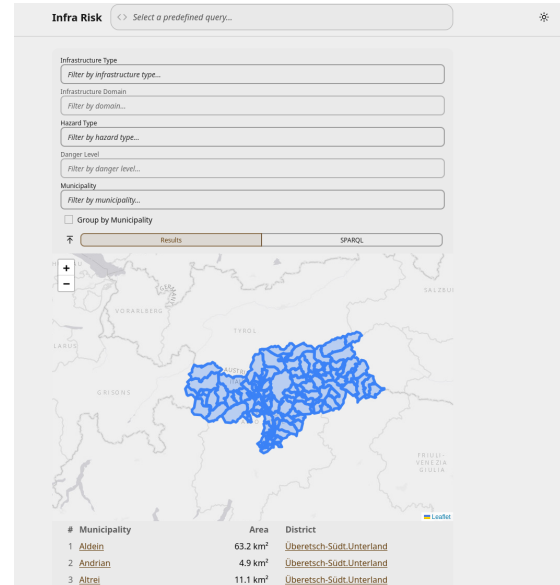
4.3.1 Main Functionalities

Preconfigured Queries The main SPARQL query used for with the Ontop endpoint is generated dynamically based on the configurations made by the user. A small number of queries has been predefined, each serving as the answer to a concrete question related to the chosen domain. Note that these are simply configuring the exact same options that are also made available directly to the user, and do not include any special cases. Figure 5a shows how these can be selected from the home screen of the application. When inside any other screen, the predefined queries can still be accessed easily with the search bar located in the application header.

Manual Configuration Possibilities However, it is not necessary to restrict oneself to the predefined queries. Figure 5b show the set of available configurable values. The first option field allows selecting the type of infrastructure to consider, lines or nodes. The second relates to the infrastructure domain, possibly filtering based on water, energy, waste, communication, or electricity related assets. Similarly to the infrastructure, also the hazards have a configurable field for whether to include avalanches, landslides, both, or neither. For the hazard zones a fourth configuration field allows selecting the danger levels to consider in the queries. Finally, it is possible to configure only a subset of municipalities to analyze, and optionally to aggregate the result by municipality. The latter is useful for answering questions about the number of vulnerable infrastructure assert



(a) Example screenshot of the landing page of the application. In the very center of the page there is a search bar with which the user can select one of the predefined configurations.



(b) Screenshot of the application returning the result for a simple query listing municipalities. Below the header, which contains the search bar for selecting other predefined queries, the application lists a number of configuration options.

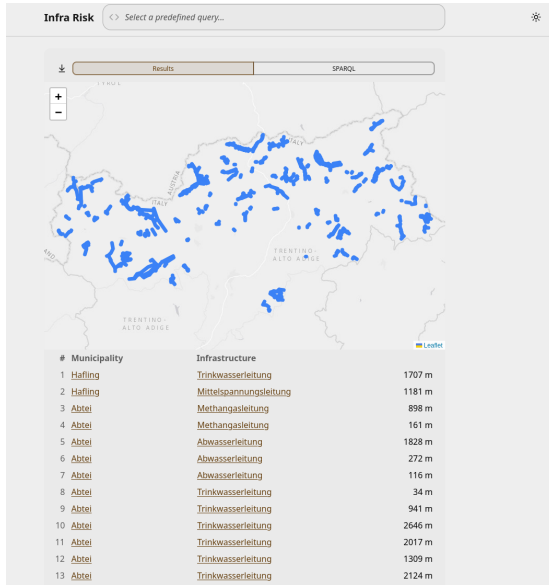
Figure 5: Screenshot of the application showing how it can be configured manually or using predefined queries.

per municipality. These configuration options, when combined, are sufficient for implementing all four example application questions that have been formulated in Section 1. For example, by not selecting any hazard type, the query is transformed into one that returns all infrastructure assets regardless of exposure. Similarly by not selecting any infrastructure type, the query is adapted to one that instead searches for hazard zones instead of infrastructure types. Removing all types for both infrastructure and hazard results in a query simply retrieving all municipalities, corresponding to what is shown in Fig. 5b. As soon as the user modifies some options, a new query is generated from it and executed against the Ontop endpoint to update the user interface as soon as the results have been received. A loading animation is shown while the application awaits the reception of the results to the latest query.

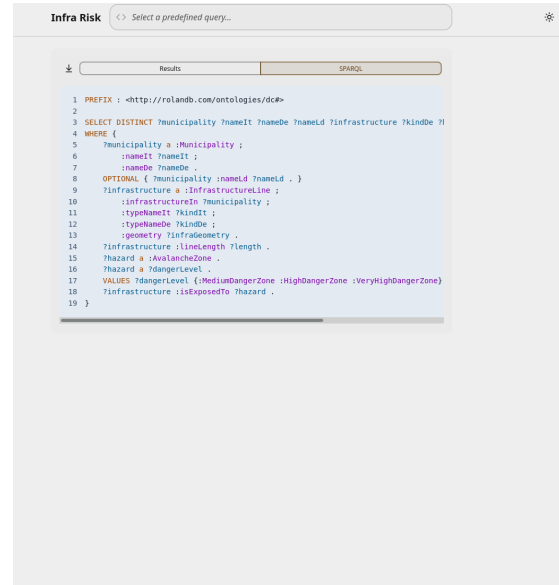
Display of Query Results Once a preconfigured query has been chosen, or a custom configuration has been created manually, the results of evaluating the resulting query over the dataset is shown below the configuration fields. For example, Fig. 6a shows the results of a query that retrieves all infrastructure lines that are exposed to avalanche hazard zones with a danger level of medium or above. Note that in Fig. 6a the configuration section of the application has been hidden using the small arrow on the top left. The results section contains two main parts. First, it contains a map that depict the geographic location of all infrastructure nodes found in the result. Additionally, a paginated table of all results is shown directly below the map. The table contains different information depending on the configuration of the query. In any case it contains some a column for the municipality and then also for either the infrastructure or hazard zone entity. The columns give a textual description of the entity which also serves as a link. Following these entity links in the table will open a dedicated page for that entity alone, as will be explained below. Note that one can select one of the geometries depicted in the map by clicking on it. Doing so will move into view and highlight the corresponding row in the table. Conversely, it is also possible to select one of the rows of the table, which will then highlight the connected geometry on the map.

Inspection of Underlying SPARQL Queries In addition to the results of the query, the application also provides the functionality of inspecting the underlying SPARQL query that has been executed against knowledge graph using Ontop. Figure 6b show an example of such a query being shown inside the application. The user can use the button labeled "SPARQL" in the top right of the screen to switch to this view, which shows the SPARQL query with syntax highlighting applied. Note that the query that is shown will dynamically update in case configuration options are modified.

Pages Dedicated to Individual Entities While the ability to perform these queries into the exposure of infrastructure assets represents the main functionality of the application, it also provides small screens for

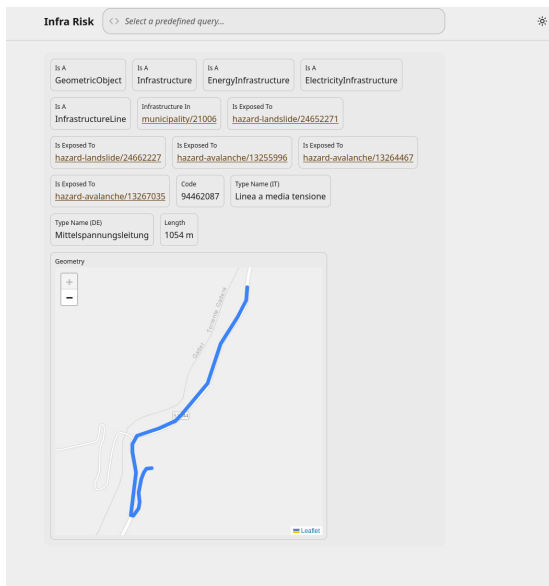


(a) Screenshot of the application showing the results for a more complicated query involving the discovery of vulnerable infrastructure lines. The results of the currently configured query are displayed both as a map and as a table.

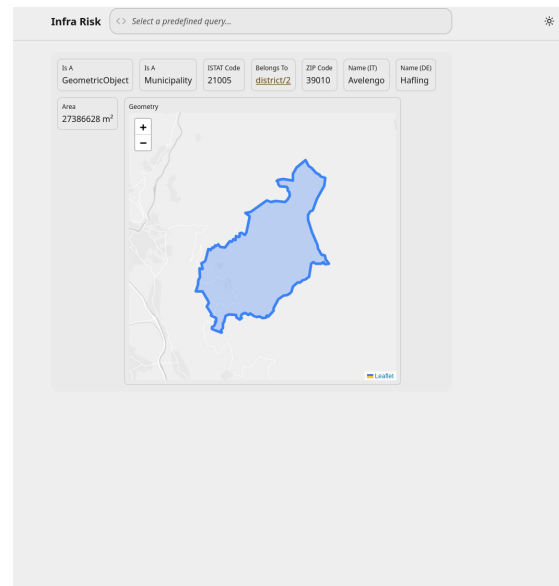


(b) Screenshot of the application showing the ability to display the SPARQL query that is being used to retrieve the results given the current configuration of the application.

Figure 6: Example screenshot extracted from the application showing how results are displayed.



(a) Application screenshot that shows details regarding one specific infrastructure line.



(b) Screenshot of the application showing the details for a single municipality.

Figure 7: Screenshot extracted from the application showing the dedicated pages related to individual entities.

inspecting individual entities. When the result of a query contains one of municipalities, infrastructure nodes or line, hazard zones, or districts, then this entity is accompanied by a link to a screen that give more information about it. Note that this information is also retrieved using SPARQL from the Ontop endpoint like all other data shown in the application. Figure 7a depicts an example of such a screen for an infrastructure line. As can be seen, in these situations the application shows information such all the classes that the entity belongs to, all the object properties including links to the target entity, as well as all data properties. The geometry data property is handled specially, by actually rendering the geometry on a small map. Figure 7b shows a similar screen but this time for a municipality. The same sort of information is shown, and the geometry is again plotted on a map.

5 Conclusion

In conclusion, the project successfully implemented a complete data profiling and curation pipeline to analyze infrastructure exposure to natural hazards. The work started with Elementary Data Analysis of five administrative and spatial datasets, utilizing the YData Profiling library alongside the Metanome data profiling platform. By performing unique column combination, functional dependency, and inclusion dependency discovery algorithms, essential metadata was extracted and used to guide the subsequent relational schema design process. This profiling led to a highly normalized database schema that eliminated source data redundancies, stripped irrelevant attributes, and enforced semantic data integrity. The resulting database was then loaded into a PostGIS database instance.

Following the data preparation and schema design phase, the chosen infrastructure and hazards domain was modeled conceptually with an OWL 2 QL ontology. This ontology was represented using the graphical notation introduced in the course and implemented directly in Protégé. To connect this ontology to the underlying relational database, Virtual Knowledge Graph mappings were developed using the Ontop Plugin for Protégé. These mappings leveraged SQL joins and spatial PostGIS functions, such as `ST_Intersects()`, to dynamically assert topological relationships and class memberships.

Finally, a custom single-page web application was developed, allowing users to query the Ontop SPARQL endpoint interactively. This end-to-end solution achieves the project’s analytical goals by seamlessly answering queries regarding the vulnerability of critical infrastructure across different hazard zones and municipalities.

References

- [1] Diego Calvanese. *DATA PREPARATION AND INTEGRATION: Part 4. Virtual Knowledge Graphs for Data Integration*. Course slides for Free University of Bolzano course number 73056. Accessed August 23, 2026. Winter 2025/2026.
- [2] Diego Calvanese, Benjamin Cogrel, Sarah Komla-Ebri, Roman Kontchakov, Davide Lanti, Martin Rezk, Mariano Rodriguez-Muro, and Guohui Xiao. “Ontop: Answering SPARQL queries over relational databases”. In: *Semantic Web* 8.3 (2016), pp. 471–487.
- [3] Aidan Hogan. “SPARQL query language”. In: *The Web of Data*. Springer, 2020, pp. 323–448.
- [4] *Information technology — Database languages — SQL multimedia and application packages — Part 3: Spatial*. Standard ISO/IEC 13249-3:2016. Geneva, Switzerland: International Organization for Standardization and International Electrotechnical Commission, 2016. URL: <https://www.iso.org/standard/60317.html>.
- [5] Meta Platforms, Inc. *React Documentation*. <https://react.dev/>. Accessed: 2026-01-07. 2026.
- [6] “OWL 2 Web Ontology Language Profiles (Second Edition)”. In: (2012). Ed. by Boris Motik, Bernardo Cuenca Grau, Ian Horrocks, Zhe Wu, Achille Fokoue, and Carsten Lutz. URL: <https://www.w3.org/TR/2012/REC-owl2-profiles-20121211/>.
- [7] Mark A Musen. “The protégé project: a look back and a look forward”. In: *AI matters* 1.4 (2015), pp. 4–12.
- [8] Thorsten Papenbrock, Tanja Bergmann, Moritz Finke, Jakob Zwiener, and Felix Naumann. “Data Profiling with Metanome”. In: *Proc. VLDB Endow.* 8.12 (Aug. 2015), pp. 1860–1863. ISSN: 2150-8097. DOI: 10.14778/2824032.2824086. URL: <http://dx.doi.org/10.14778/2824032.2824086>.
- [9] Thorsten Papenbrock, Sebastian Kruse, Jorge-Arnulfo Quiané-Ruiz, and Felix Naumann. “Divide & conquer-based inclusion dependency discovery”. In: *Proceedings of the VLDB Endowment* 8.7 (2015), pp. 774–785.
- [10] Thorsten Papenbrock and Felix Naumann. “A hybrid approach for efficient unique column combination discovery”. In: *Datenbanksysteme für Business, Technologie und Web (BTW 2017)*. Gesellschaft für Informatik, Bonn. 2017, pp. 195–204.
- [11] Thorsten Papenbrock and Felix Naumann. “A hybrid approach to functional dependency discovery”. In: *proceedings of the 2016 International Conference on Management of Data*. 2016, pp. 821–833.
- [12] PostGIS Development Group. *PostGIS: Spatial and Geographic Objects for PostgreSQL*. Version 3.6. Open Source Geospatial Foundation (OSGeo). 2026. URL: <https://postgis.net>.
- [13] Shopify, Inc. *React Router Documentation*. <https://reactrouter.com/>. Accessed: 2026-01-07. 2026.
- [14] VoidZero Inc. *Vite Documentation*. <https://vitejs.dev/guide/>. Accessed: 2026-01-07. 2026.
- [15] YData Labs Inc. *YData Profiling Documentation*. <https://docs.profiling.ydata.ai/latest/>. Accessed: 2026-02-10. 2026.